Módulo 2: API de pólizas (backend y frontend)

API REST en Spring Boot 3.5 (Java 17) y SPA en Angular 21 para gestionar pólizas de arrendamiento (individuales y colectivas) y sus riesgos: consulta, creación, renovación con IPC y cancelación. Cada operación que cambia el estado de una póliza o de un riesgo se notifica al sistema CORE.

Respuesta corta. El backend es la fuente de verdad de las reglas de negocio y las concentra en el modelo de dominio; el frontend solo presenta, valida formato y deshabilita acciones. Ambos se levantan con un solo `docker compose up --build`. Está cubierto con 96 pruebas de backend, 80 unitarias de frontend y 10 e2e.

Documentos detallados: backend/README.md · frontend/README.md. Las decisiones técnicas, con su alternativa descartada, están en backend/DECISIONES.md y frontend/DECISIONES.md.

Cómo ejecutarlo

Requiere Docker Desktop:

```
cd 02-api-polizas
docker compose up --build
```

Servicio	URL
Frontend	http://localhost:4200
API (Swagger UI)	http://localhost:8080/swagger-ui.html
API key de prueba	`123456` (header `x-api-key`)

Si los puertos están ocupados: `WEB_PORT=4300 API_PORT=8082 docker compose up --build`. Para detener: `docker compose down`.

También se puede ejecutar cada parte por separado (backend con H2 en memoria, `./mvnw spring-boot:run`; frontend con `npm ci && npm start`); los pasos están en cada README.

Endpoints

Método	Ruta	Descripción	Éxito
GET	`/polizas?tipo=&estado=`	Lista pólizas (filtros opcionales)	200
GET	`/polizas/{id}`	Consulta una póliza	200
POST	`/polizas`	Crea una póliza con sus riesgos	201
GET	`/polizas/{id}/riesgos`	Lista los riesgos de una póliza	200
POST	`/polizas/{id}/renovar`	Renueva aplicando el IPC	200
POST	`/polizas/{id}/cancelar`	Cancela la póliza y todos sus riesgos	200
POST	`/polizas/{id}/riesgos`	Agrega un riesgo (solo COLECTIVA)	201
POST	`/riesgos/{id}/cancelar`	Cancela un riesgo	200
GET	`/parametros/renovacion`	IPC vigente que se aplicará al renovar	200
POST	`/core-mock/evento`	Mock del CORE: registra el evento en logs	202

Reglas de negocio (viven en el dominio)

- Una póliza **INDIVIDUAL** tiene exactamente un riesgo; una **COLECTIVA** admite cero o más.
- **Prima = canon × meses de vigencia**, con `BigDecimal` y redondeo `HALF_EVEN`. Nunca `double`.
- **Renovar** aplica el IPC al canon (`canon × (1 + IPC)`), recalcula la prima y extiende la vigencia; una póliza cancelada no se renueva.
- **Cancelar** una póliza cancela también todos sus riesgos; cancelar algo ya cancelado es un conflicto.
- Agregar un riesgo a una **INDIVIDUAL** es una regla de negocio violada (422); a una póliza **cancelada** es un conflicto de estado (409).
- Toda modificación de estado de pólizas o riesgos invoca el CORE a través del puerto `CorePort`.

Arquitectura (hexagonal ligera)

Capa	Contenido
`domain/`	Entidades con las reglas, excepciones de negocio, eventos de dominio y puertos. Sin dependencias de web ni de Spring (salvo anotaciones JPA).
`application/`	Casos de uso (`PolizaService` , `RiesgoService`): cargan, invocan el dominio, guardan y notifican al CORE.
`infrastructure/`	Web (controllers, DTOs, mappers, errores), persistencia JPA, adaptador del CORE, notificaciones, seguridad y configuración.

Errores y observabilidad

Todas las respuestas de error siguen RFC 9457 – Problem Details e incluyen un `correlationId` para buscar la traza en los logs.

Código	Cuándo
400	Campos inválidos (con detalle por campo), JSON mal formado o parámetros con tipo inválido
401	API key ausente o incorrecta
404	La póliza, el riesgo o la ruta no existe
409	Conflicto con el estado actual (cancelar algo ya cancelado, renovar una cancelada, modificación concurrente)
422	Regla de negocio violada por la petición (riesgo en una INDIVIDUAL, INDIVIDUAL sin exactamente un riesgo)

Cada petición recibe un `correlationId` que se devuelve en la respuesta, aparece en los logs y se propaga al CORE. Los logs registran solo identificadores: nunca nombres de tomadores, direcciones ni la API key.

Frontend

SPA Angular 21 (standalone, *zoneless* y *signals*) que consume la API a través del proxy `/api` (el navegador nunca conoce la URL del backend).

- **Listado** con filtros por tipo y estado (los filtros van en la URL, se pueden compartir).
- **Detalle** con datos, vigencia, canon, prima y tabla de riesgos; acciones según el estado y el tipo.
- **Creación** de INDIVIDUAL (un riesgo) y COLECTIVA (`FormArray` , 0..n) con prima estimada y errores 400 del backend en cada campo.
- **Renovación** en un diálogo que muestra el IPC y compara los valores anteriores frente a los nuevos.
- **Cancelación** de póliza y de riesgo con confirmación (`<dialog>` nativo accesible).

Estados de carga, vacío y error en cada vista; accesibilidad obligatoria (etiquetas, `aria-*` , contraste AA).

Pruebas

Ámbito	Herramienta	Cantidad
Backend (dominio, servicios, repositorio, integración con MockMvc)	JUnit 5, Mockito, JaCoCo	96 pruebas · 99,8 % de líneas y 91 % de ramas
Frontend unitarias	Vitest	80 pruebas
Frontend e2e	Playwright (Chromium)	10 flujos sobre la aplicación real

El build del backend **falla** si la cobertura de líneas baja del 80 %. La colección de Postman (backend/postman/) incluye 20 peticiones con aserciones. El detalle de cada suite está en el README correspondiente.

Documentos relacionados

- backend/README.md · backend/DECISIONES.md
- frontend/README.md · frontend/DECISIONES.md
- Diseño de la solución y trazabilidad: ../01-diseno-sistema/ARQUITECTURA.md